

Análisis del Pipeline ETL hacia Snowflake

Avance 3 – Proyecto Integrador

Introducción

Este documento presenta el diseño, ejecución y validación del proceso ETL que integra la base relacional de FleetLogix en PostgreSQL con un Data Warehouse en Snowflake.

Arquitectura General

El pipeline implementa una arquitectura híbrida entre el entorno local y la nube, con los siguientes componentes:

- **Origen:** base de datos PostgreSQL, con las tablas transaccionales vehicles, drivers, routes, trips y deliveries.
- **Transformación:** script 05_ETL_Pipeline.py, desarrollado en Python utilizando librerías **pandas**, **SQLAlchemy** y **snowflake-connector-python**.
- **Destino:** esquema analítico en Snowflake, compuesto por las tablas DIM_DATE, DIM_TIME, DIM_DRIVER, DIM_VEHICLE, DIM_ROUTE, DIM_CUSTOMER y FACT_DELIVERIES.
- **Frecuencia:** carga incremental de la última semana, determinada dinámicamente según el valor máximo de delivered_datetime en PostgreSQL.

Esta arquitectura permite que la información operativa se integre de forma incremental y escalable en la nube, lista para ser explotada en análisis históricos o dashboards de rendimiento.

Diseño y Ejecución del Pipeline

Extracción dinámica (ancla de tiempo y ventana móvil):

- *En SQL:* El proceso identifica automáticamente el punto de anclaje temporal y define una ventana móvil de siete días para extracción:

```
SELECT MAX(delivered_datetime)
FROM deliveries;
```

- *En Python:* Mediante pandas y SQLAlchemy, se extraen los datos de interés:

```
trips = pd.read_sql(
    f"SELECT * FROM trips WHERE
    departure_datetime >= '{since}'",
    pg
)
deliveries = pd.read_sql(
    f"SELECT * FROM deliveries WHERE
    delivered_datetime >= '{since}'",
    pg
)
```

Esto asegura que solo se carguen entregas recientes o modificadas, optimizando el rendimiento y evitando duplicados.

Transformación a dimensiones

Se generan dimensiones de tiempo, fecha, clientes, conductores, vehículos y rutas.

Ejemplo de normalización para DIM_DATE y DIM_TIME:

```
dim_date['date_key'] = (
    pd.to_datetime(dim_date['full_date'])
    .dt.strftime('%Y%m%d')
    .astype(int)
)
dim_time['time_key'] = sorted({
    *[int(pd.Timestamp(x).strftime('%H%M'))
      for x in
    deliveries['scheduled_datetime'].dropna()],
    *[int(pd.Timestamp(x).strftime('%H%M'))
      for x in deliveries['delivered_datetime'].dropna()],
})
```

Estas transformaciones permiten construir claves surrogate y mantener una granularidad homogénea para el análisis en el modelo estrella.

Construcción de tabla de hechos

La tabla de hechos (FACT_DELIVERIES_STG) combina información de entregas y viajes.

Durante el proceso se asegura la correcta vinculación con las dimensiones:

```
fact_stg = (
    deliveries.merge(
        trips,
        on='trip_id',
        suffixes=('_del', '_trip')
    )
)
fact_stg['date_key'] = (
    fact_stg['delivered_datetime']
    .dt.strftime('%Y%m%d')
    .astype(int)
)
fact_stg = fact_stg.merge(
    dim_customer[['customer_id', 'customer_key']],
    on='customer_id',
    how='left'
)
```

El resultado es una tabla staging normalizada y preparada para ser insertada en Snowflake.

Carga en Snowflake (staging + insert final)

La carga se realiza en dos etapas: primero hacia la tabla temporal `FACT_DELIVERIES_STG` mediante la función `write_pandas`, y luego hacia la tabla final `FACT_DELIVERIES`:

```
ok, _, nrows, _ = write_pandas(
    con,
    df2,
    'FACT_DELIVERIES_STG',
    quote_identifiers=False
)

cur.execute("""
    INSERT INTO FACT_DELIVERIES (...)
    SELECT ...
    FROM FACT_DELIVERIES_STG;
""")
```

Esta estrategia permite mantener un control intermedio de las cargas y facilita auditorías o reprocesos parciales.

Métricas del Proceso

Resultados de carga

Tabla	Filas
DIM_DATE	8
DIM_TIME	1,246
DIM_DRIVER	400
DIM_VEHICLE	200
DIM_ROUTE	50
DIM_CUSTOMER	3,841
FACT_DELIVERIES	3,889

Estos valores se corresponden con la ejecución exitosa del pipeline, confirmada mediante el archivo de log `etl_last_week.log`.

Validaciones de Calidad

Durante el proceso se ejecutaron controles automáticos de integridad y consistencia, incluyendo:

- **Integridad temporal:**
Verificación de que todas las entregas cumplan `delivered_datetime ≥ scheduled_datetime`.

```
assert (fact_stg['delivered_datetime'] >= fact_stg['scheduled_datetime']).all()
```
- **Integridad referencial:**
Chequeo de correspondencia entre las claves foráneas del hecho y las dimensiones (`customer_key`, `driver_key`, `vehicle_key`, `route_key`).
- **Consistencia de conteos:**
Comparación de totales entre origen y destino, asegura == de registros.

- **Registro detallado de logs:**

Cada fase (extracción, validación, carga) se documenta en `etl_last_week.log`, incluyendo tiempos y conteos procesados.

Estas validaciones garantizan la calidad y confiabilidad de los datos antes de su análisis en Snowflake.

Evidencias de Verificación

Las siguientes consultas realizadas en Snowflake fueron utilizadas para comprobar la correcta carga de datos en el esquema

`FLEETLOGIX_DW.BI_SCHEMA`:

```
SHOW TABLES IN SCHEMA
FLEETLOGIX_DW.BI_SCHEMA;

SELECT TABLE_NAME, ROW_COUNT
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'BI_SCHEMA';

SELECT COUNT(DISTINCT CUSTOMER_KEY),
COUNT(*)
FROM FACT_DELIVERIES;
```

Además, se visualizó una muestra de la tabla `FACT_DELIVERIES` para confirmar la presencia de todas las claves y métricas esperadas.

Conclusiones y Próximos Pasos

El pipeline ETL diseñado para FleetLogix demostró un funcionamiento estable y eficiente, logrando integrar de manera incremental la información operativa al Data Warehouse de Snowflake.

El modelo estrella quedó completamente disponible para análisis analítico y exploraciones de rendimiento logístico.